

Constitutional Commonwealth — JC Compute

Authority-Constrained Deterministic Causal Computation

The Commonwealth = $(H, C, R, \pi, \sqcup)$

History · Capability · Reducer · Projection · Ratification

— subject to deterministic replay and convergence invariants.

What This Is

The Constitutional Commonwealth is a formal computational model in which administration is defined as **replayable reduction over causally ordered proceeding histories**, bounded by **capability authority**.

A commonwealth is a self-governing system of citizens operating under shared constitutional rules. No citizen holds privilege over another. No territory can escalate beyond its chartered authority. All jurisdictions converge to the same ratified record — regardless of the order in which proceedings arrive.

This repository contains:

Layer	Contents
Formal foundations <code>((formal/))</code>	Machine-checked proofs (T3-T6) + model-checked specs (T1-T2)
Core substrate <code>((packages/))</code>	UniStack v3 — Field, Reducer, IR, Merkle, Great Seal, Sync, Query, Meta, Runtime
Runtime <code>((runtime/))</code>	Ratification operator F_Q , oracle, CRDT bill merger, deliberation phase space
Application layer <code>((app/))</code>	CLI + HTTP server; 6 territory demos; full legislative pipeline
Tests	20+ unit and integration test files
Benchmarks	13 benchmark suites

Constitutional Theorems

All six theorems from *JC Compute Formal Foundations* are machine-checked:

#	Theorem	Tool	File
T1	Deterministic Replay	TLA+ / TLC	<code>formal/tla/DeterministicReplay.tla</code>
T2	Convergence	TLA+ / TLC	<code>formal/tla/Convergence.tla</code>
T3	Capability Preservation	Lean 4 / Mathlib	<code>formal/lean/CapabilityPreservation.lean</code>
T4	Commutativity Characterisation	Lean 4 / Mathlib	<code>formal/lean/CommutativityCharacterization.lean</code>
T5	Projection Consistency	Lean 4 / Mathlib	<code>formal/lean/ProjectionConsistency.lean</code>
T6	Fixed-Point Existence	Lean 4 / Mathlib	<code>formal/lean/FixedPointExistence.lean</code>

Verify the Lean Proofs

```
bash

# Requires elan (Lean version manager)
lake update    # downloads Mathlib once
lake build     # type-checks all proofs – zero sorry, zero axioms beyond Mathlib
```

Run the TLA+ Model Checker

```
bash

java -cp tla2tools.jar tlc2.TLC formal/tla/DeterministicReplay.tla -config formal/tl
java -cp tla2tools.jar tlc2.TLC formal/tla/Convergence.tla          -config formal/tl
```

Quick Start (JavaScript Runtime)

```
bash
```

```

npm install

# CLI demo
node src/cli.js --help
node src/cli.js listTerritories
node src/cli.js administer --domain financial-settlement

# Application server (port 3000)
npm run app:start

# Tests
npm test

# Benchmarks
npm run bench:kcr
npm run bench:frontier

```

Application CLI

```

bash

node app/src/cli.js listTerritories
node app/src/cli.js administer --domain distributed-robotics
node app/src/cli.js administer --domain ai-agent-os --intent converge --steps
node app/src/cli.js unionConvergence --domains distributed-robotics,financial-settle
node app/src/cli.js conductLegislativeProcess --domain multi-agent-swarms --runs 10

```

Architecture

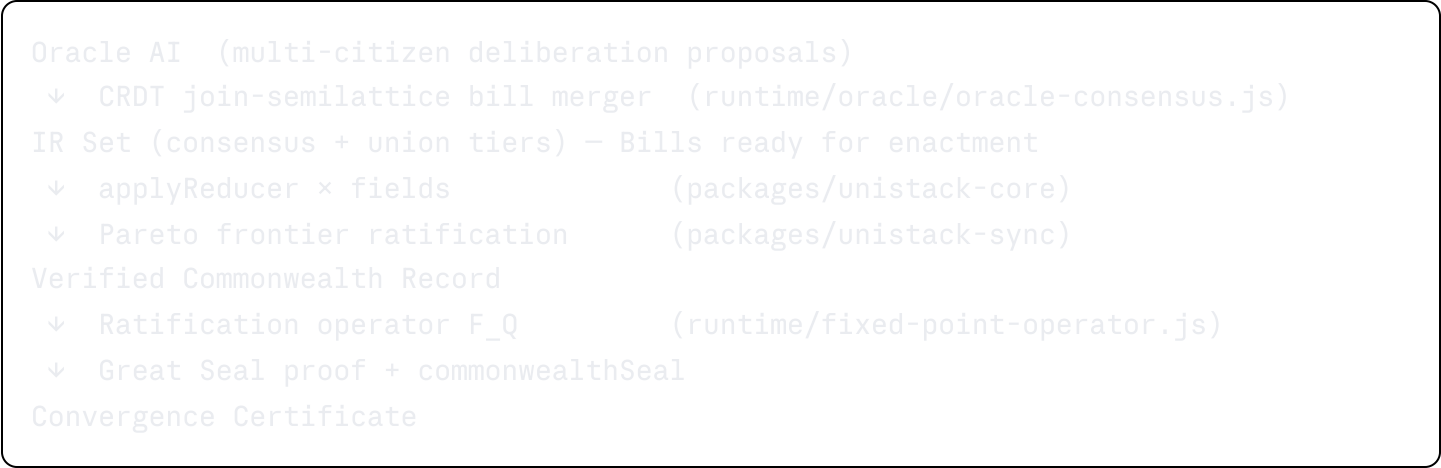
```

Constitutional Commonwealth
(H, C, R,  $\pi$ ,  $\sqcup$ )

    History (Proceedings)
      |
    Reduction (R) — Reducer administers IR
      |
Commonwealth Record — Projection ( $\pi$ ) — View
      |
    Ratify ( $\sqcup$ ) ← Capability (C)

```

The Legislative Pipeline (UniStack)



Packages

Package	Constitutional Role
unistack-core	Field algebra, reducer administration, IR rulings, Merkle, Great Seal primitives
unistack-sync	Ratification, Pareto frontier, jurisdiction synchronization, evidence proofs
unistack-query	Jurisdictional reachability, membership evidence, frontier petitions, ratification analysis
unistack-meta	Electoral search over reducers, merges, and architectures; dominance ordering
unistack-runtime	Civil service scheduler, administration loop, public registry, citizen node execution
unistack-store	Public record archive — persistent commonwealth state

Territories (Application Layer)

Each territory is a domain with its own charter, founding state, and reducer rules. All territories are subject to the same constitutional invariants.

Territory	ID
 Distributed Robotics	<code>distributed-robotics</code>
 AI Agent OS	<code>ai-agent-os</code>
 Financial Settlement	<code>financial-settlement</code>
 Autonomous Manufacturing	<code>autonomous-manufacturing</code>
 Multi-Agent Swarms	<code>multi-agent-swarms</code>
 Digital Twin	<code>digital-twin</code>

```
js
import { foundTerritory, getTerritoryCharter, listTerritories } from "./vocabulary.js"

// Found a new territory with a given width × height field and initial state
const territory = foundTerritory({ width: 4, height: 4 });

// Retrieve the charter (config) for a named territory
const charter = getTerritoryCharter("financial-settlement");

// List all known territories
const all = listTerritories();
```

Core API Reference

Citizens — `unistack-runtime`

A **citizen** is a participating node in the commonwealth. Each citizen holds a local state (a set of fields), a reducer rule, and a topology describing its relationships to other citizens.

```
js
```

```
import { createCitizen, administer, civilService, publicRegistry } from "../vocabulary"

// Found a citizen with a 4x4 field, seeded with initial cells
const citizen = createCitizen({
  width: 4,
  height: 4,
  initialCells: [],
  reducerSpec: { kind: "standard-v1" },
  irSpec: { kind: "u8-payload4" },
  topology: { peers: [] }
});

// Administer a set of IR rulings to a citizen – returns the updated citizen
const updated = administer(citizen, [ruling1, ruling2]);

// The civil service coordinates administration across all citizens in one step
const nextGeneration = civilService([citizen1, citizen2], [inboxA, inboxB]);

// Inspect the public registry of available layers
const registry = publicRegistry();
// → { core: "unistack-core", sync: "unistack-sync", ... }
```

The Great Seal — **unistack-core**

The **Great Seal** (Merkle root) is the canonical cryptographic commitment to a citizen's entire field state. Any change to any cell in any field changes the Great Seal.

js

```
import {
  seal,
  greatSeal,
  commonwealthRecord,
  constitutionalSeal,
  sealID
} from "./vocabulary.js";

// Seal any value — produces a 64-char BLAKE2b-512 hex digest
const digest = seal({ opcode: 0, payload: [1, 2, 3, 4] });

// Compute the Great Seal for a single field
const fieldCommitment = greatSeal(citizenField);

// Compute the commonwealth record root for an entire state (set of fields)
const stateCommitment = commonwealthRecord(citizenState);

// Compute the full constitutional seal for a citizen object
// (commits state, reducer, IR spec, topology, merge rule, and spec version)
const citizenRoot = constitutionalSeal(citizen);

// Derive a stable ID from any root value
const id = sealID(citizenRoot);
```

Deliberation & Rulings — **unistack-core**

Deliberation generates IR proposals (rulings) — deterministic instruction records that control how reducers transform state. **Enacting** applies a consensus IR set to produce the next state.

```
js
```

```

import {
  deliberate,
  holdElection,
  enactRuling,
  administerUntilRatified,
  conductLegislativeProcess
} from "./vocabulary.js";

// Deliberate: generate IR proposals for a given state
const proposals = deliberate(currentState, queryFamily);

// Hold an election over a set of proposals – returns the consensus IR set
const electionResult = holdElection(proposals, citizens);

// Enact: apply the consensus IR to advance the commonwealth record
const nextRecord = enactRuling(currentRecord, electionResult);

// Administer until ratified: run reduction until the fixed point is reached
const ratified = administerUntilRatified(initialState, irStream, query);

// Conduct the full legislative process end-to-end
const certificate = conductLegislativeProcess(config);

```

An **IR ruling** has the minimal shape:

```

js

{ opcode: 0,           // u8 – operation selector
  payload: [0,0,0,0] // uint256[4] – operands }

```

The opcode determines how the reducer transforms each cell. Identical rulings produce identical transformations on every citizen machine.

Ratification — `unistack-sync`

Ratification is the lattice-preserving operator for knowledge accumulation. Two branches of the commonwealth record are ratified into one. The result is always at least as advanced as either branch.

Ratification satisfies three constitutional properties:

- **Idempotence:** `ratify(A, A) = A`

- **Commutativity:** $\text{ratify}(A, B) = \text{ratify}(B, A)$
- **Associativity:** $\text{ratify}(A, \text{ratify}(B, C)) = \text{ratify}(\text{ratify}(A, B), C)$

```
js

import {
  ratify,
  propagateProceedings,
  compareSeals,
  findConstitutionalConflict,
  generateEvidence,
  applyAmendment,
  commonwealthSeal,
  synchronizeJurisdictions
} from "./vocabulary.js";

// Ratify two state branches into one – returns the Pareto-frontier union
const unified = ratify(branchA, branchB);

// Propagate proceedings: apply incoming rulings to a citizen and ratify
const advanced = propagateProceedings(citizen, incomingRulings);

// Compare two commonwealth seals for equality
const comparison = compareSeals(sealLeft, sealRight);
// → { equal: true|false, left, right }

// Detect where two jurisdictions have diverged
const conflict = findConstitutionalConflict(treeLeft, treeRight);
// → { diverged: bool, path: [...], left, right }

// Generate a cryptographic evidence proof for a state
const evidence = generateEvidence({ root, witness, merklePath, queryType });

// Apply an amendment (delta) to local state – equivalent to ratify(local, delta)
const amended = applyAmendment(localState, delta);

// Compute the commonwealth seal for a citizen (commits all protocol parameters)
const seal = commonwealthSeal(citizen);

// Synchronize two jurisdictions using delta-based evidence exchange
const synced = synchronizeJurisdictions(localCitizen, remoteCitizen);
```

Evidence & Testimony — **unistack-sync**

The commonwealth issues cryptographic **evidence** (Merkle proofs) and **testimony** (witness paths) to verify that a ruling or membership claim is valid without replaying all proceedings.

```
js

import {
  evidenceFor,
  testimonyFor,
  verifyTestimony,
  containsEvidence,
  membershipEvidence,
  amendmentEvidence,
  conflictEvidence
} from "./vocabulary.js";

// Generate evidence that a root is present in the state tree
const proof = evidenceFor(stateTree, targetRoot);

// Generate a witness path for a specific node
const witness = testimonyFor(stateTree, nodeIndex);

// Verify that testimony is authentic for a given seal
const valid = verifyTestimony(greatSeal, testimony);

// Check if evidence for a root is contained in the state tree
const present = containsEvidence(stateTree, evidenceProof);

// Prove membership of a field in the current commonwealth record
const membership = membershipEvidence(stateTree, fieldRoot);

// Produce evidence that an amendment was validly applied
const amendment = amendmentEvidence(before, after, ruling);

// Produce evidence of a constitutional conflict between two branches
const conflict = conflictEvidence(branchA, branchB);
```

Frontier Petitions — **unistack-query**

Petitions query the frontier of the commonwealth record — what is reachable, what is constitutionally required, and whether ratification has been achieved.

js

```
import {
  jurisdictionallyReachable,
  constitutionallyRequired,
  mustReachRatification,
  legislativeFrontier,
  adjudicationEvidence,
  mayProceed,
  frontierPetitions,
  ratificationReached
} from "./vocabulary.js";

// Test whether a target state is jurisdictionally reachable from source
const reachable = jurisdictionallyReachable(sourceState, targetState);

// Assert that a ruling must happen under all valid legislative paths
const required = constitutionallyRequired(state, ruling);

// Assert that the state must eventually reach ratification
const guaranteed = mustReachRatification(state, query);

// Compute the legislative frontier – all states one ruling away
const frontier = legislativeFrontier(state, rulingSet);

// Produce adjudication evidence for a query result
const evidence = adjudicationEvidence(queryResult);

// Check whether a proceeding may proceed under current capability rules
const allowed = mayProceed(state, proceeding);

// Issue a batch of frontier petitions and return results
const results = frontierPetitions(state, petitions);

// Test whether the fixed point (ratification) has been reached
const done = ratificationReached(currentState, previousState);
```

The Ratification Operator — `runtime/fixed-point-operator.js`

The **ratification operator** (F_Q) is the heart of the runtime. It applies a ruling, ratifies the result, projects through the query family, and compresses — then repeats until the commonwealth record stabilises.

```

js

import { createRatificationOperator } from "../vocabulary.js";

const F_Q = createRatificationOperator({
  queryProjector: (state, query) => { /* filter state to query-relevant fields */ }
});

// Advance state by one ruling under query Q
const next = F_Q.step(currentState, ruling, query);

// Apply ruling: reduce → ratify → project → compress
const applied = F_Q.apply(currentState, ruling, query);

// Run to fixed point — administer until commonwealth record stops changing
const ratified = F_Q.fix(initialState, rulingStream, query, limit=32);

// Compose two query families for cross-domain proceedings
const combined = F_Q.composeQueries(queryA, queryB);

```

The operator is **robust** when $(F_Q(S, \text{adversarialRulings}) == F_Q(S, \text{nominalRulings}))$. If they diverge, a province boundary has been found.

Bill Merger (CRDT Consensus) — `runtime/oracle/oracle-consensus.js`

Multiple citizens (or oracle models) propose bills (IR proposals). The **CRDT bill merger** combines all proposals into a single IR set using a join-semilattice — commutative, associative, and idempotent.

```

js

```

```

import { mergeBillsCRDT } from "./vocabulary.js";

const { irSet, consensus, union, scores } = mergeBillsCRDT(
  [
    { model: "oracle-alpha", proposals: [bill1, bill2] },
    { model: "oracle-beta", proposals: [bill2, bill3] }
  ],
  {
    consensusThreshold: 2, // bills endorsed by ≥ 2 models enter the consensus tier
    maxIR: 8               // maximum bills in the resulting IR set
  }
);

// irSet - all bills ordered by weight (consensus tier first, union tier second)
// consensus - bills endorsed by ≥ threshold models
// union - bills endorsed by exactly one model
// scores - Map of fingerprint → { ir, weight, models }

```

Tier structure:

- **Consensus tier** — bills present in $\geq$ `consensusThreshold` citizens → highest authority
- **Union tier** — bills from exactly one citizen → enters record with weight 1

Ordering within each tier is deterministic: weight descending, opcode ascending. The final IR set is identical regardless of the order in which citizens submit.

The Opposition — `runtime/adversarial-ir-generator.js`

The **opposition** stress-tests the commonwealth by generating adversarial rulings — boundary conditions, contradictions, entropy injections, and replays of known-bad proceedings. If the ratification operator withstands opposition pressure, the commonwealth is robust.

js

```
import { createOpposition } from './vocabulary.js';

const opposition = createOpposition({ oracleConfig });

// Generate both cooperative and adversarial ruling streams
const { nominal_irSet, adversarial_irSet } = await opposition.generate(currentState,

// Robustness test: the commonwealth holds if both streams reach the same fixed point
const nominalResult      = F_Q.fix(state, nominal_irSet, query);
const adversarialResult = F_Q.fix(state, adversarial_irSet, query);
const robust = JSON.stringify(nominalResult) === JSON.stringify(adversarialResult);
```

Adversarial strategies:

Strategy	What It Tests
<code>boundary:zero</code> / <code>boundary:maxint</code>	Whether the reducer saturates or wraps gracefully
<code>boundary:neg</code>	Negative payload handling
<code>entropy:alternating</code>	Structure preservation under high-entropy input
<code>entropy:pseudorandom</code>	Deterministic chaos via the golden angle
Contradiction IRs	Merge commutativity under conflicting proposals
Replay IRs	Idempotence under known-bad historical proceedings
Null IRs	Identity stress — opcode 0, all-zero payload

Deliberation Phase Space — `runtime/compression-phase-space.js`

The deliberation phase space maps the dynamics of the commonwealth record across time — how provinces cluster, how proceedings couple, and how the system responds to constitutional shocks.

js

```
import {
  deliberationVector,
  deliberationDistance,
  clusterDistricts,
  provinceConsistencyMap,
  proceedingsCoupling,
  constitutionalShockResponse,
  constitutionalShockMatrix,
  provinceTransitionProbability,
  provinceTransitionStatistics
} from "./vocabulary.js";

// Compute the deliberation phase vector for a state history
const vector = deliberationVector(stateHistory);

// Measure the deliberation distance between two phase vectors
const dist = deliberationDistance(vectorA, vectorB);

// Cluster states into districts (attractor basins) by phase similarity
const districts = clusterDistricts(phaseVectors, k=3);

// Map consistency within each province (how stable is each attractor?)
const consistency = provinceConsistencyMap(stateTrajectory);

// Measure how tightly two proceedings trajectories are coupled
const coupling = proceedingsCoupling(trajjectoryA, trajectoryB);

// Measure system response to a constitutional shock (perturbation)
const response = constitutionalShockResponse(state, perturbation);

// Full perturbation matrix: response to all perturbations
const matrix = constitutionalShockMatrix(state, perturbations);

// Probability that the commonwealth transitions from one province to another
const prob = provinceTransitionProbability(provinceA, provinceB, history);

// Full transition statistics across all provinces
const stats = provinceTransitionStatistics(history);
```

Quotient Space — `runtime/quotient-space.js`

The quotient space measures the **diversity of the public record** and the **geometry of jurisdictions**. These are optimization diagnostics — not constitutional preconditions — that help characterise how efficiently the commonwealth compresses knowledge.

```
js

import {
  publicRecordDiversity,
  publicRecordDiversityFromCounts,
  jurisdictionalDistance,
  jurisdictionalGeometry,
  unifiedConstitutionalInvariant,
  sealDivergence,
  constitutionalMetrics
} from "../vocabulary.js";

// Entropy of the public record – how diverse are the representative states?
const entropy = publicRecordDiversity(states);
const entropyFromCounts = publicRecordDiversityFromCounts(countMap);

// Distance between two states within the same jurisdiction class
const dist = jurisdictionalDistance(stateA, stateB);

// Full within-class geometry for a set of states
const geometry = jurisdictionalGeometry(stateSet);

// The unified constitutional invariant – collapses all quotient metrics to one value
const invariant = unifiedConstitutionalInvariant(stateSet, queryFamily);

// Divergence between two commonwealth seals (root-level distance)
const divergence = sealDivergence(rootA, rootB);

// Full suite of constitutional metrics for a state set
const metrics = constitutionalMetrics(stateSet, queryFamily);
// → { entropy, withinClassDistance, canonicalizationRate, KCR, ... }
```

Electoral Search — `unistack-meta`

The meta layer conducts **electoral searches** over the space of possible reducers, merges, and

architectures. Candidates are evaluated, ranked by merit, and evolved (amended) toward constitutional optimality.

```
js

import {
  judiciallyReview,
  orderByMerit,
  conductElectoralSearch,
  amendCandidates,
  searchConstitutionalIDs,
  rankConstitutionalIDs
} from "./vocabulary.js";

// Judicially review a candidate – score it against the query family
const score = judiciallyReview(candidate, testSuite);

// Order a set of candidates by constitutional merit
const ranked = orderByMerit(candidates, testSuite);

// Conduct an electoral search over the candidate space
const winner = conductElectoralSearch({
  initialPopulation: seeds,
  queryFamily,
  generations: 20
});

// Amend candidates – evolve the population toward better solutions
const nextGeneration = amendCandidates(candidates, { mutationRate: 0.1 });

// Search for constitutional IDs matching a query
const matchingIDs = searchConstitutionalIDs(queryFamily, corpus);

// Rank constitutional IDs by merit score
const orderedIDs = rankConstitutionalIDs(ids, testSuite);
```

The Public Record Archive — **unistack-store**

The **archive** persists the commonwealth record across sessions. Each entry is addressable by its filing address (root hash).

```
js
```

```
import { createArchive, filingAddress } from "./vocabulary.js";

// Create the public record archive
const archive = createArchive();

// Compute the filing address (root hash) for an archived record
const address = filingAddress(archive);
```

Cross-Domain Union Convergence

Multiple territories can be administered together. `unionConvergence` runs the full legislative process across all domains simultaneously, ratifying a shared commonwealth record.

```
js

import { unionConvergence } from "./vocabulary.js";

const sharedRecord = unionConvergence({
  territories: ["distributed-robotics", "financial-settlement"],
  query: myQueryFamily,
  steps: 10
});
```

Constitutional Properties

The commonwealth upholds these guarantees by construction — not by policy:

- **No randomness** — fully deterministic administration
- **No clocks** — time-independent proceedings
- **No ordering assumptions** — convergent under any delivery order of rulings
- **No consensus layer** — convergence is structural via Pareto ratification
- **No privilege escalation** — formally impossible (T3, C3)

A citizen who receives the same rulings as another will always arrive at the same commonwealth record. A jurisdiction that ratifies with another will always produce the same Great Seal. The fixed point exists and is unique (T6).

Knowledge Compression Ratio (KCR)

KCR is the metric for query-preserving compression of the public record.

Compression is constitutionally valid only if it preserves the relevant petition family:

$$q(S) = q(\text{compressQ}(S)) \quad \forall q \in Q$$

If semantic preservation holds:

$$KCR = |S| / |S \sim Q|$$

Otherwise: $KCR = 0$. A ruling that cannot be semantically compressed without changing petition outcomes must be preserved in full.

The evaluation harness records: `workload`, `policy`, `preservation`, `compressionRatio`, `KCR`, `proofSize`, `syncCost`.

Vocabulary Reference

Constitutional Term	Technical Term	Module
<code>seal</code>	<code>hash</code>	core
<code>greatSeal</code>	<code>fieldRoot</code>	core
<code>commonwealthRecord</code>	<code>stateRoot</code>	core
<code>constitutionalSeal</code>	<code>rootOf</code>	core
<code>sealID</code>	<code>rootID</code>	core
<code>deliberate</code>	<code>generateIRProposals</code>	core
<code>holdElection</code>	<code>runConsensus</code>	core
<code>enactRuling</code>	<code>applyConsensusIR</code>	core
<code>administerUntilRatified</code>	<code>runToFixedPoint</code>	core
<code>conductLegislativeProcess</code>	<code>runFullPipeline</code>	core

Constitutional Term	Technical Term	Module
<code>foundTerritory</code>	<code>createDomainState</code>	core
<code>getTerritoryCharter</code>	<code>getDomainConfig</code>	core
<code>listTerritories</code>	<code>listDomains</code>	core
<code>unionConvergence</code>	<code>runCrossDomainConvergence</code>	core
<code>createCitizen</code>	<code>createNode</code>	runtime
<code>administer</code>	<code>execute</code>	runtime
<code>civilService</code>	<code>scheduler</code>	runtime
<code>publicRegistry</code>	<code>layerRegistry</code>	runtime
<code>createGovernmentFixedPoint</code>	<code>createRuntimeFixedPointOperator</code>	runtime
<code>createArchive</code>	<code>createStore</code>	store
<code>filingAddress</code>	<code>rootAddress</code>	store
<code>ratify</code>	<code>merge</code>	sync
<code>propagateProceedings</code>	<code>syncStep</code>	sync
<code>compareSeals</code>	<code>compareRoots</code>	sync
<code>findConstitutionalConflict</code>	<code>findDivergence</code>	sync
<code>generateEvidence</code>	<code>generateProof</code>	sync
<code>applyAmendment</code>	<code>applyDelta</code>	sync
<code>commonwealthSeal</code>	<code>syncRoot</code>	sync
<code>synchronizeJurisdictions</code>	<code>deltaSync</code>	sync
<code>evidenceFor</code>	<code>proofFor</code>	sync
<code>testimonyFor</code>	<code>witnessFor</code>	sync
<code>verifyTestimony</code>	<code>verifyWitness</code>	sync
<code>jurisdictionallyReachable</code>	<code>reachable</code>	query

Constitutional Term	Technical Term	Module
constitutionallyRequired	mustHappen	query
mustReachRatification	mustReach	query
legislativeFrontier	frontier	query
adjudicationEvidence	proof	query
mayProceed	canHappen	query
frontierPetitions	frontierQueries	query
ratificationReached	fixpointReached	query
judiciallyReview	evaluate	meta
orderByMerit	rank	meta
conductElectoralSearch	search	meta
amendCandidates	evolve	meta
createRatificationOperator	createFixedPointOperator	fixed-point
mergeBillsCRDT	mergeProposalsCRDT	oracle-consensus
createOpposition	createAdversarialGenerator	adversarial
deliberationVector	compressionPhaseVector	phase-space
deliberationDistance	cpsDistance	phase-space
clusterDistricts	clusterCpsVectors	phase-space
provinceConsistencyMap	basinConsistencyMap	phase-space
proceedingsCoupling	trajectoryCoupling	phase-space
constitutionalShockResponse	perturbationResponse	phase-space
constitutionalShockMatrix	perturbationMatrix	phase-space
provinceTransitionProbability	basinTransitionProbability	phase-space

Constitutional Term	Technical Term	Module
<code>publicRecordDiversity</code>	<code>representativeEntropy</code>	quotient-space
<code>jurisdictionalDistance</code>	<code>withinClassDistance</code>	quotient-space
<code>jurisdictionalGeometry</code>	<code>withinClassGeometry</code>	quotient-space
<code>unifiedConstitutionalInvariant</code>	<code>unifiedQuotientInvariant</code>	quotient-space
<code>sealDivergence</code>	<code>rootDivergenceDistance</code>	quotient-space
<code>constitutionalMetrics</code>	<code>quotientMetrics</code>	quotient-space

Licensing

- Personal / non-commercial: `LICENSE-PERSONAL.md`
- Commercial / organizational: `LICENSE-COMMERCIAL.md` + `LICENSING.md`